ASSIGNMENT-1

```
import pandas as pd

import nltk

nltk.download('punkt_tab')


para="Natural Language Processing is also with 3 letter called NLP. NLP is the most loving subject, me 2. it's
include playing with words"

from nltk.tokenize import word_tokenize


tokens=word_tokenize(para)

print(tokens)


from nltk.tokenize import sent_tokenize

sent=sent_tokenize(para)

print(sent)


from nltk.tokenize import wordpunct_tokenize

print(wordpunct_tokenize(para))


from nltk.tokenize import TreebankWordTokenizer

tokenizer = TreebankWordTokenizer()

tree_tokens=tokenizer.tokenize(para)

print(tree_tokens)


from nltk.stem import PorterStemmer

stemmer=PorterStemmer()

for word in tree_tokens:

    print(word, "---->", stemmer.stem(word))
```

```
from nltk.stem import RegexpStemmer

reg_stem = RegexpStemmer('ing$|s$|e$|able$')
word=['eating','writes','readable','running']
for w in word:
    print(w,"-->", reg_stem.stem(w))

from nltk.stem import SnowballStemmer
snowball =SnowballStemmer("english")
for word in tree_tokens:
    print(word ,"-->", snowball.stem(word))
print("example")
print("fairly -->", snowball.stem("fairly"))

import nltk
nltk.download('wordnet')

from nltk.stem import WordNetLemmatizer
lemmatizer=WordNetLemmatizer()
for word in tree_tokens:
    print(word,"-->",lemmatizer.lemmatize(word, pos='v'))

#from nltk.stem import WordNetlemmatizer
lemmatizer=WordNetLemmatizer()
word2=["eating","eats","eaten","writing","writes","programming"]
for word in word2:
    print(word, "-->", lemmatizer.lemmatize(word, pos="v"))

import nltk
```

```
nltk.download('stopwords')

from nltk.corpus import stopwords
from nltk.stem import PorterStemmer
stemmer=PorterStemmer()
sentence=nltk.sent_tokenize(para)
for i in range(len(sentence)):
    words=nltk.word_tokenize(sentence[i])
    words=[stemmer.stem(word) for word in words if word.lower() not in set(stopwords.words('english'))]
    sentence[i]=" ".join(words)
    print(sentence)

import re
from num2words import num2words
def convert_number_to_words(text):
    return re.sub(r"\d+", lambda x: num2words(int(x.group())),text)
result = convert_number_to_words(para)
print("\n number to words using library: \n")
print(result)

num_dict = {
        "0": "zero",
        "1": "one",
        "2": "two",
        "3": "three",
        "4": "four",
        "5": "five",
        "6": "six",
```

```
        "7": "seven",
        "8": "eight",
        "9": "nine"
}
def num_to_words_simple(text):
    for digit,words in num_dict.items():
        text=text.replace(digit, words)
    return text
print("\n number to words without library\n")
print(num_to_words_simple(para))
```

ASSIGNMENT -2

```
def min_edit1(str1,str2):
    m = len(str1)
    n = len(str2)

    T = []
    for i in range(m+1):
        row = []
        for j in range(n+1):
            row.append(0)
        T.append(row)

    for i in range(m+1):
        for j in range(n+1):

            # For null
            if i==0 and j==0:
```

```
            T[i][j] = 0
        # for first row
        elif i == 0:
            T[i][j] = T[i][j-1] + 1
        # for first Col
        elif j == 0:
            T[i][j] = T[i-1][j] + 1
        #if row == col
        elif str1[i-1] == str2[j-1]:
            T[i][j] = T[i-1][j-1]
        # if Row != Col
        else:
            T[i][j] = min(T[i][j-1], T[i-1][j], T[i-1][j-1]) + 1
print("Matrix==",T)
i, j = m, n
operations = []

while i > 0 or j > 0:

    # Match
    if i > 0 and j > 0 and str1[i-1] == str2[j-1]:
        operations.append("Match")
        i -= 1
        j -= 1
    # Replace
    elif i > 0 and j > 0 and T[i][j] == T[i-1][j-1] + 1:
        operations.append("Replace")
        i -= 1
        j -= 1
```

```
        # Delete
        elif i > 0 and T[i][j] == T[i-1][j] + 1:
            operations.append("Delete")
            i -= 1
        # Insert
        else:
            operations.append("Insert")
            j -= 1

    operations.reverse()

    return T[m][n], operations, T
dist, ops, matrix = min_edit1("money", "monkey")

for row in matrix:
    print(row)
print("Edit Distance:", dist)
print("Edit Operations Only:")
print("Replace:", ops.count("Replace"))
print("Insert :", ops.count("Insert"))
print("Delete :", ops.count("Delete"))
```

ASSIGNMENT-3

```
def viterbi_algorithm(obs, states, pi, trans, emit):

    v = [{}]
    path = {}
```

```
# Initialization
for state in states:
    v[0][state] = pi[state] * emit[state][obs[0]]
    path[state] = [state]

best_state = max(v[0], key=v[0].get)
print("After 1 word (", obs[0], ") ->", best_state)

# Recursion
for t in range(1, len(obs)):
    v.append({})
    new_path = {}

    for curr_state in states:
        max_prob = 0
        best_prev = None

        for prev_state in states:
            prob = (
                v[t-1][prev_state] *
                trans[prev_state][curr_state] *
                emit[curr_state][obs[t]]
            )

            if prob > max_prob:
                max_prob = prob
                best_prev = prev_state
```

```
            v[t][curr_state] = max_prob
            new_path[curr_state] = path[best_prev] + [curr_state]

        path = new_path

        best_state = max(v[t], key=v[t].get)
        print("After word", t+1, "(", obs[t], ") ->", best_state)
        print("Probability Table:", v)

    final_state = max(v[-1], key=v[-1].get)
    final_seq = path[final_state]

    print("\nFinal Best State Sequence:", final_seq)

states = ['N', 'M', 'V']

obs = ['will', 'will', 'google', 'campus']

pi = {'N': 0.6, 'M': 0.4, 'V': 0.0}

trans = {
    'N': {'N': 0.1, 'M': 0.1, 'V': 0.8},
    'M': {'N': 0.8, 'M': 0.1, 'V': 0.1},
    'V': {'N': 0.8, 'M': 0.1, 'V': 0.1}
}
emit = {
    'N': {'nitish': 0.2, 'loves': 0.0001, 'campus': 0.3, 'google': 0.0001, 'will': 0.2, 'ankita': 0.2, 'can': 0.0001},
    'M': {'nitish': 0.0001, 'loves': 0.0001, 'campus': 0.0001, 'google': 0.0001, 'will': 0.5, 'ankita': 0.0001, 'can': 0.5},
    'V': {'nitish': 0.0001, 'loves': 0.6, 'campus': 0.0001, 'google': 0.4, 'will': 0.0001, 'ankita': 0.0001, 'can': 0.0001}
```

```
}


viterbi_algorithm(obs, states, pi, trans, emit)
```

ASSIGNMENT- 4

```
# Step 1: Import
import pandas as pd
import numpy as np
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Embedding, LSTM, TimeDistributed, Dense
from sklearn.model_selection import train_test_split

# Step 2: Load & Clean Data
df = pd.read_csv("NLP Datasets/NER dataset.csv", encoding="latin1")
df.head()

df["Sentence #"] = df["Sentence #"].ffill()
df["Word"] = df["Word"].bfill()

# Step 3: Create Sentences
sentences = df.groupby("Sentence #")["Word"].apply(list).values
tags = df.groupby("Sentence #")["POS"].apply(list).values

# Step 4: Tokenization
```

```
tok = Tokenizer()
tok.fit_on_texts(sentences)
X = tok.texts_to_sequences(sentences)

tag_tok = Tokenizer()
tag_tok.fit_on_texts(tags)
y = tag_tok.texts_to_sequences(tags)

# Step 5: Padding
MAX_LEN = 128
X = pad_sequences(X, maxlen=MAX_LEN, padding='post')
y = pad_sequences(y, maxlen=MAX_LEN, padding='post')


# Step 6: Train-Test Split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)

# Step 7: Model
model = Sequential([
    Embedding(len(tok.word_index)+1, 64),
    LSTM(64, return_sequences=True),
    TimeDistributed(Dense(np.max(y)+1, activation='softmax'))
])

model.compile(optimizer='adam', loss='sparse_categorical_crossentropy', metrics=['accuracy'])


# Step 8: Train
model.fit(X_train, y_train, epochs=5, batch_size=32)
```

```
# Step 9: Evaluate
loss, acc = model.evaluate(X_test, y_test)
print("Accuracy:", acc)

# Step 10: Prediction
sample = sentences[0]
seq = pad_sequences(tok.texts_to_sequences([sample]), maxlen=MAX_LEN, padding='post')

pred = np.argmax(model.predict(seq), axis=-1)[0]

idx2tag = {v:k for k,v in tag_tok.word_index.items()}

for w, t in zip(sample, pred[:len(sample)]):
    print(w, "->", idx2tag.get(t))
```

ASSIGNMENT-5

```
!pip install spacy
!python -m spacy download en_core_web_sm

import spacy

nlp = spacy.load("en_core_web_sm")

def simple_coref(text):
    doc = nlp(text)
```

```
    resolved = []
    last_noun = None

    for token in doc:
        if token.pos_ in ["NOUN", "PROPN"]:
            last_noun = token.text
            resolved.append(token.text)

        elif token.pos_ == "PRON" and last_noun:
            resolved.append(last_noun)

        else:
            resolved.append(token.text)

    return " ".join(resolved)

def simple_core(text):
    txt=nlp(text)
    resolved=[]
    last_noun=None

    for token in txt:
        if token.pos_ in ['NOUN','PROPN']:
            last_noun=token.text
            resolved.append(token.text)

        elif token.pos_ == 'PRON' and last_noun:
            resolved.append(last_noun)
```

```
        else:
            resolved.append(token.text)

    return " ".join(resolved)



texts = input("Enter text: ")
ans=simple_core(text)
print(ans)




#texts=nlp(texts)
#for t in texts:
#    print(t)

#text=nlp(text)
#for txt in text:
#    print(txt)
#    print(txt.text,"-->",txt.pos_)

resolved_text = simple_coref(text)

print("\nOriginal Text:")
print(text)

print("\nResolved Text:")
print(resolved_text)
```

```
### without spaCy or any pretrained model

pronouns = {
    "he", "she", "it", "they", "him", "her", "them",
    "his", "hers", "its", "their"
}
def simple_coref(text):
    words = text.split()

    resolved = []
    last_noun = None

    for word in words:
        clean_word = word.strip(".,!?").lower()

        if word[0].isupper() and clean_word not in pronouns:
            last_noun = word
            resolved.append(word)

        elif clean_word in pronouns and last_noun:
            resolved.append(last_noun)

        else:
            resolved.append(word)

    return " ".join(resolved)

text = input("Enter text: ")
```

```
resolved_text = simple_coref(text)
print(resolved_text)

#for word in text:
    #clean_word = word.strip(".,!?").lower()
    #print(word.strip(".,!?"))

resolved_text = simple_coref(text)

print("\nOriginal Text:")
print(text)

print("\nResolved Text:")
print(resolved_text)
Enter text:  John went to the store. He bought milk.
```

ASSIGNMENT-6

```
import nltk
from nltk.corpus import wordnet
from nltk.wsd import lesk

nltk.download('wordnet')
nltk.download('omw-1.4')

s1 = "I went to the bank to deposit money"
s2 = "He sat on the bank of the river"
```

```
words1 = s1.split()
words2 = s2.split()

# Lesk senses
sense1 = lesk(words1, "bank")
sense2 = lesk(words2, "bank")
# Function to calculate overlap score
def overlap_score(context, sense):
    sense_words = set(sense.definition().lower().split())
    context_words = set(context)
    return len(sense_words.intersection(context_words))

# Compute scores
score1 = overlap_score(words1, sense1)
score2 = overlap_score(words2, sense2)

# OUTPUT
print("Sentence 1 Sense:", sense1)
print("Meaning:", sense1.definition())
print("Overlap Score:", score1)

print("\nSentence 2 Sense:", sense2)
print("Meaning:", sense2.definition())
print("Overlap Score:", score2)

# Word Sense Disambiguation using WordNet (Improved Lesk Algorithm)

# Step 1: Import libraries
```

```
import nltk
nltk.download('punkt')
nltk.download('wordnet')
nltk.download('stopwords')

from nltk.corpus import wordnet as wn
from nltk.tokenize import word_tokenize
from nltk.corpus import stopwords

# Step 2: Define stopwords
stop_words = set(stopwords.words('english'))

# Step 3: Improved Lesk Function
def improved_lesk(sentence, word):
    # Tokenize sentence and remove stopwords
    context = set(word_tokenize(sentence.lower()))
    context = context - stop_words

    best_sense = None
    max_overlap = 0

    # Loop through all noun senses of the word
    for sense in wn.synsets(word, pos=wn.NOUN):

        # Get definition tokens
        signature = word_tokenize(sense.definition().lower())

        # Add example tokens
        for example in sense.examples():
```

```
            signature += word_tokenize(example.lower())

        # Remove stopwords from signature
        signature = set(signature) - stop_words

        # Calculate overlap
        overlap = len(context.intersection(signature))

        # Select best sense
        if overlap > max_overlap:
            max_overlap = overlap
            best_sense = sense

    return best_sense


# Step 4: Test Sentences
sentence1 = "I went to the bank to deposit money in my account"
sentence2 = "The river overflowed the bank after heavy rain"

# Step 5: Apply WSD
sense1 = improved_lesk(sentence1, "bank")
sense2 = improved_lesk(sentence2, "bank")

# Step 6: Print Results
print("Sentence 1:", sentence1)
print("Predicted Sense:", sense1)
print("Definition:", sense1.definition())
```

```
print("\nSentence 2:", sentence2)
print("Predicted Sense:", sense2)
print("Definition:", sense2.definition())
```

ASSIGNMENT-7

```
pip install gensim nltk pandas

# Step 1: Load Dataset
import json

data = []
with open("Cell_Phones_and_Accessories_5.json", "r", encoding="utf-8") as f:
    for line in f:
        data.append(json.loads(line))

print("Dataset size:", len(data))

# Extract review text
sentences = [item['reviewText'] for item in data if 'reviewText' in item]
print(sentences[10])

# Step 3: Preprocessing
import nltk, re
from nltk.tokenize import word_tokenize
from nltk.corpus import stopwords
```

```
nltk.download('punkt')
nltk.download('stopwords')

stop_words = set(stopwords.words('english'))

def preprocess(text):
    text = re.sub(r'[^a-z\s]', '', text.lower())
    words = word_tokenize(text)
    return [w for w in words if w not in stop_words]

tokenized_sentences = [preprocess(s) for s in sentences if s]

print(tokenized_sentences[1])

# Step 4: Train Word2Vec
from gensim.models import Word2Vec

w2v_model = Word2Vec(
    sentences=tokenized_sentences,
    vector_size=100,
    window=5,
    min_count=5
)

# Similar words
print("\nWord2Vec Similar Words:")
print(w2v_model.wv.most_similar("movie"))

# Similarity
```

```
print("Similarity (phone, mobile):",
      w2v_model.wv.similarity("phone", "mobile"))

# Step 5: Train FastText
from gensim.models import FastText

ft_model = FastText(
    sentences=tokenized_sentences,
    vector_size=100,
    window=5,
    min_count=5
)

print("\nFastText Similar Words:")
print(ft_model.wv.most_similar("phone"))

print("Similarity (phone, mobile):",
      ft_model.wv.similarity("phone", "mobile"))
```

ASSIGNMENT-8

```
# Step 1: Import
import pandas as pd
import re, nltk
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize
from nltk.stem import WordNetLemmatizer

from sklearn.model_selection import train_test_split
```

```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, classification_report

nltk.download('punkt')
nltk.download('stopwords')
nltk.download('wordnet')

# Step 2: Load Data (ONLY train.csv)
df = pd.read_csv("AG_news(train).csv")
df.head()

# Step 3: Combine text (important)
df["text"] = df["Title"] + " " + df["Description"]

# Step 4: Preprocessing
stop_words = set(stopwords.words('english'))
lemma = WordNetLemmatizer()

def clean(text):
    text = re.sub(r'[^a-z\s]', '', text.lower())
    words = word_tokenize(text)
    return " ".join([lemma.lemmatize(w) for w in words if w not in stop_words])

df["clean"] = df["text"].apply(clean)

# Step 5: Split data
```

```
X_train, X_test, y_train, y_test = train_test_split(
    df["clean"], df["Class Index"], test_size=0.2, random_state=42
)

# Step 6: TF-IDF
vec = TfidfVectorizer(max_features=5000)
X_train = vec.fit_transform(X_train)
X_test = vec.transform(X_test)

nb = MultinomialNB()
nb.fit(X_train, y_train)

nb_pred = nb.predict(X_test)
nb_acc = accuracy_score(y_test, nb_pred)

print("Naive Bayes Accuracy:", nb_acc)
print(classification_report(y_test, nb_pred))

lr = LogisticRegression(max_iter=200)
lr.fit(X_train, y_train)

lr_pred = lr.predict(X_test)
lr_acc = accuracy_score(y_test, lr_pred)

print("Logistic Regression Accuracy:", lr_acc)
print(classification_report(y_test, lr_pred))

rf = RandomForestClassifier()
rf.fit(X_train, y_train)
```

```
rf_pred = rf.predict(X_test)
rf_acc = accuracy_score(y_test, rf_pred)

print("Random Forest Accuracy:", rf_acc)
print(classification_report(y_test, rf_pred))

import matplotlib.pyplot as plt

models = ["Naive Bayes", "Logistic Regression", "Random Forest"]
acc = [nb_acc, lr_acc, rf_acc]

plt.bar(models, acc)

# Step 11: Sample Prediction (Categorization)
labels = {1:"World", 2:"Sports", 3:"Business", 4:"Sci/Tech"}

sample = "Stock market rises due to tech growth"
sample_clean = clean(sample)

pred = lr.predict(vec.transform([sample_clean]))[0]
print("Predicted Category:", labels[pred])
```

ASSIGNMENT-9

```
import pandas as pd

df = pd.read_csv("winequality_main.csv")
df.head()
```

```
print(df.shape)
print(df.columns)

X = df.drop("quality", axis=1)

from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

from sklearn.cluster import AgglomerativeClustering

model = AgglomerativeClustering(n_clusters=3)
labels = model.fit_predict(X_scaled)

print(set(labels))

print("Number of clusters:", len(set(labels)))

linked = linkage(X_scaled, method='complete')

plt.figure(figsize=(12,6))
dendrogram(linked, truncate_mode='lastp', p=30)
plt.title("Dendrogram - Wine Dataset")
```

ASSIGNMENT-10

```
import pandas as pd
```

```
df=pd.read_csv("part4/NLP Datasets/IMDB Dataset.csv")

from sklearn.feature_extraction.text import TfidfVectorizer

tfidf=TfidfVectorizer(stop_words="english",max_features=50000)
x=tfidf.fit_transform(df["review"])

y=df["sentiment"]

from sklearn.model_selection import train_test_split
x_train, x_test, y_train, y_test=train_test_split(x,y,train_size=0.8, random_state=42)
from sklearn.linear_model import LogisticRegression
lr=LogisticRegression()

lr.fit(x_train,y_train)

pred=lr.predict(x_test)
print(pred)

#Sentiment analysis
c_review=input("Enter the review: ")
review_vc=tfidf.transform([c_review])
pred_sentiment=lr.predict(review_vc)[0]
print(pred_sentiment)

#keyword extraction
def keywords_find(txt,top_n=5):
    vc=tfidf.transform([txt])
```

```
    score=vc.toarray()[0]

    feature_names=tfidf.get_feature_names_out()

    keywords=sorted(zip(score,feature_names),reverse=True)[:top_n]

    return [word for score,word in keywords if score>0]

keyword=keywords_find(c_review)
print(keyword)

import re
from nltk.corpus import stopwords
from nltk.stem import WordNetLemmatizer
lemmatizer=WordNetLemmatizer()

def clean_data(txt):
    if pd.isna(txt):
        return ""

    txt=re.sub(r"<.*?>","",txt)
    txt=re.sub(r"[^a-zA-Z]","",txt)

    txt=txt.lower()
    words=txt.split()

#    for word in words:
#        if word not in (set(stopwords.words("english"))):
#            words=lemmatizer.lemmatize(word)
```

```
#    words=[words]

    words = [lemmatizer.lemmatize(word)
            for word in words
            if word not in set(stopwords.words('english'))]

    return " ".join(words)

df["review_clean"]=df["review"].apply(clean_data)

def preprocess_review(text):
    if pd.isna(text):
        return ""

    # Remove HTML tags first
    text = re.sub(r'<.*?>', ' ', text)

    # Keep only alphabets
    text = re.sub(r'[^a-zA-Z]', ' ', text)

    # Lowercase
    text = text.lower()

    # Tokenize
    words = text.split()

    # Remove stopwords + lemmatize
    words = [lemmatizer.lemmatize(word)
            for word in words
```

```
        if word not in set(stopwords.words('english'))]

    return ' '.join(words)

df['clean_review'] = df['review'].apply(preprocess_review)
```